Listen

Learning Doc 10: Stage 1, the Foundations in Code
The plumbing every later part of the agent stands on

Implementation, stage 1 of 6

21 October 2026

Abstract

Docs 01–09 were design: what Listen should do and why. This is the first document about real code. Stage 1 built the agent's foundations: the package and its command line, settings, logging, events, the event bus, the worker loop every module runs, the supervisor that watches the workers, and CI. None of it hears your voice yet; all of it will be used by every later stage. Each section shows the actual code, the Python ideas it uses, and why it was written that way.

Contents

1 What stage 1 built **2**

2 Installing your own package **2**
2.1 The virtual environment 2
2.2 `pyproject.toml`, section by section 2

3 The command line **3**
3.1 Exit codes 4

4 Settings: a file the program trusts only after checking **4**

5 Frozen dataclasses **4**

6 Threads and queues **5**

7 The event bus: publish and subscribe **6**

8 Locks and race conditions **7**

9 The supervisor **7**

10 Dependency injection and fakes **8**

11 pytest tools used in stage 1 **8**

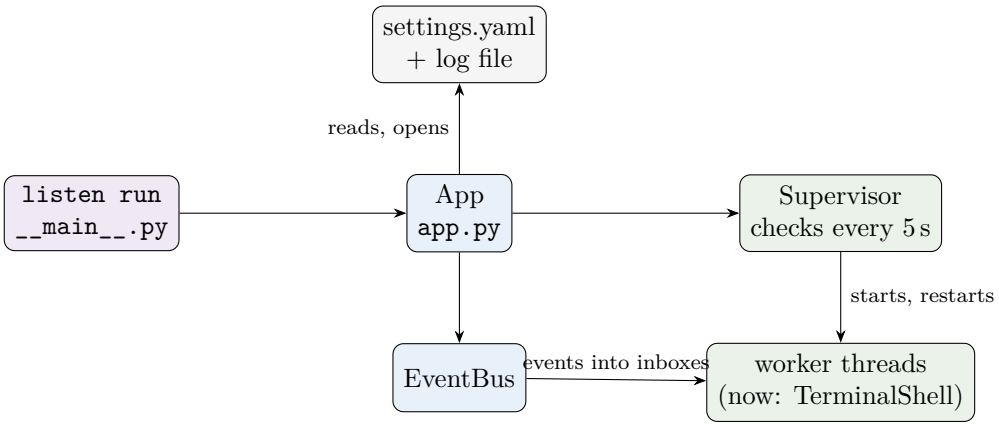
12 Logging **9**

13 ruff and CI **9**

14 Glossary **9**

15 Check your understanding **10**

1 What stage 1 built



What happens when you type `listen run`:

1. The log file is opened (`agent/logs/listen.log`).
2. `agent/settings.yaml` is read and checked. A bad value stops here with a message naming the key, exit code 2.
3. The `App` creates the event bus, the workers (in stage 1 only the terminal shell) and the supervisor, and starts one thread per worker plus one for the supervisor.
4. "Listen is ready." is printed and logged. The main thread waits.
5. Ctrl+C: "Stopping…", every thread is told to stop, "stopped" is logged, exit code 0.

Why it is done this way

Every one of the 11 modules in stage 2–4 needs settings, logging, a thread, an inbox and a way to talk to the others. Building that once, first, and testing it on its own means each later module only has to write its own job. It is also pure Python with no microphone or AI model, which makes it the gentlest place to learn the tools.

2 Installing your own package

2.1 The virtual environment

```
python -m venv agent/.venv
agent/.venv/Scripts/python -m pip install -e "agent[dev]"
```

The first line creates `agent/.venv/`: a private copy of Python with its own library folder (Doc 02). The second installs the agent *into* it:

- `-e` (**editable**): the installed package *points at* your `agent/listen_agent/` folder instead of copying it. Edit a file, run again, the change is there. Without `-e` you would reinstall after every edit.
- `[dev]`: an **extra**, an optional group of libraries from `pyproject.toml`. `pytest` and `ruff` are needed to develop, not to run Listen, so they live in the `dev` group.

2.2 `pyproject.toml`, section by section

```
[project]
name = "listen-agent"
dynamic = ["version"]                    # read from the code, see below
requires-python = ">=3.11"
dependencies = ["pyyaml>=6.0"]           # needed to RUN
```

```
[project.optional-dependencies]
dev = ["pytest>=8.0", "ruff>=0.6"]        # needed to DEVELOP

[project.scripts]
listen = "listen_agent.__main__:main"     # creates listen.exe

[tool.setuptools.dynamic]
version = { attr = "listen_agent.__version__" }
```

`[project.scripts]` is why typing `listen` works: pip writes a small `listen.exe` into `.venv/Scripts/` that calls `main()` in `listen_agent/__main__.py`. The version is written once, in `listen_agent/__init__.py`, and `pyproject.toml` reads it from there, so the two can never disagree.

Concept: pip + venv instead of uv

Doc 02 introduced **uv**, a faster tool that does the same job and also writes a lock file. uv is not installed on this laptop and the CI file (Doc 09) uses pip, so stage 1 uses the tools that come with Python. The idea of a lock file (exact versions of everything) is still worth having later; switching to uv then changes two commands, not the code.

Watch out

The install also creates `agent/listen_agent.egg-info/`: pip's bookkeeping, rebuilt on every install. It is now in `.gitignore`, like `.venv/`. Rule of thumb: anything a tool can regenerate does not belong in git.

3 The command line

```
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="listen", description="Listen voice agent")
    parser.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
    commands = parser.add_subparsers(dest="command", metavar="command")
    commands.add_parser("run", help="start the agent (Ctrl+C stops it)")
    return parser

def main(argv: list[str] | None = None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)
    if args.command == "run":
        return run_agent()
    parser.print_help()
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

- **`python -m listen_agent`** runs the file `__main__.py` inside the package. That is what the special file name is for.
- **`argparse`** (standard library) turns `listen run` into `args.command == "run"`, and writes the `--help` text for free. **Subcommands** (`run`, later `review`, `commands check`, `record-tests`) are how one program offers several actions, like `git commit` and `git push`.
- **`argv=None`**: normally argparse reads what you typed; tests pass a list instead, `main(["run"])`, without starting a new process.
- **`if __name__ == "__main__"`** is true only when the file is run, not when it is imported (tests import it).

3.1 Exit codes

A program ends with a number. 0 means success; anything else means "something went wrong".

Code	Name	Meaning, and what the launcher (stage 2) will do
0	`EXIT_OK`	You stopped it. Do nothing.
2	`EXIT_BAD_SETTINGS`	`settings.yaml` is wrong. Restarting cannot help: tell you and stop.
3	`EXIT_FATAL`	A thread crash-looped or hung. Restart the agent (max 3 in 10 min).

4 Settings: a file the program trusts only after checking

```
thresholds:                          # placeholders until calibrated (spec 3.6)
  hearing: 0.60
  match: 0.75
```

`load_settings()` reads this with PyYAML's `safe_load` (which builds plain dicts, lists, strings and numbers, never arbitrary Python objects) and then checks everything: unknown keys, wrong types, numbers outside 0–1. A typo stops start-up:

```
Can't start: threshold: unknown setting
Can't start: thresholds.match: must be a number from 0 to 1, got True
```

Why it is done this way

The friendly-looking alternative, "ignore what you don't understand and use defaults", is the dangerous one. If `threshold:` (missing `s`) were ignored, your change would silently do nothing and you would spend an evening wondering why. Fail loudly, early, and say exactly where.

Watch out

YAML's "yes" trap. In YAML, unquoted `yes`, `no`, `on` and `off` are read as `True`/`False`. And in Python, `True` is a kind of integer (`True == 1`), so `match: yes` would pass a "number from 0 to 1" check. The code refuses `bool` explicitly, and a test pins it.

The BOM. Notepad can save files with an invisible *byte-order mark* at the start. Reading with `encoding="utf-8-sig"` removes it if present. One word in the code, one test, one confusing evening avoided.

Other encodings (found by the review). Notepad can also save as "Unicode" (UTF-16) or ANSI (Windows-1252). Those bytes are not valid UTF-8, so reading them raised a raw `UnicodeDecodeError`: a traceback, exit code 1, nothing in the log, and the launcher would have restarted an agent that can never start. Now any read failure becomes `settings.yaml: can't read it (...); save it as UTF-8` with exit code 2.

5 Frozen dataclasses

```
@dataclass(frozen=True)
class Thresholds:
    hearing: float = 0.60
    match: float = 0.75
```

Concept: dataclass

`@dataclass` writes the boring parts of a class for you: `__init__` (so `Thresholds(match=0.8)` works), `__eq__` (two objects with the same values are equal, which is how the test compares the shipped file with the defaults) and a readable `__repr__`. `frozen=True` makes every field read-only after creation: `settings.mode = "x"` raises `FrozenInstanceError`.

Events use the same idea , with two extra options:

```
@dataclass(frozen=True, kw_only=True)
class Event:
    created_at: datetime = field(default_factory=_now)

@dataclass(frozen=True)
class ModuleError(Event):
    module: str
    error: str
```

- `default_factory=_now`: call `_now()` for *each* new event. Writing `= datetime.now(UTC)` instead would run once, when the class is defined, and every event would share that one time.
- `kw_only=True`: Python forbids a field without a default after one with a default. `created_at` has a default and `module` does not, so normally `ModuleError` could not exist. Keyword-only fields are moved to the end, which allows `ModuleError("voice", "no mic")`.
- **UTC**: `datetime.now(UTC)` carries its time zone. Spec rule: all stored times are UTC; convert to local time only to display.

Why it is done this way

One event object is put into several inboxes and read by several threads at once. If any thread could change it, the others would see it change under them: a bug that appears once a week and never when you look. Frozen objects make that impossible.

6 Threads and queues

Concept: thread

A **thread** is a second line of execution inside the same program. Listen will have one per module so that, for example, the transcriber can work on a recording while the listener keeps hearing. `threading.Thread(target=f).start()` runs `f` alongside everything else.

Concept: queue

A `queue.Queue` is a thread-safe waiting line: one thread `puts`, another `gets`, in order. `get(timeout=1)` waits up to one second and then raises `queue.Empty`. Every worker owns one, its **inbox**.

The worker loop, the heart of every module:

```
_NOTHING = object()

def run_once(self) -> None:
    self.beat()                                    # heartbeat: "still alive"
    try:
        message = self.inbox.get(timeout=self.wait_seconds)
    except queue.Empty:
```

```
        message = _NOTHING
    try:
        if message is _NOTHING:
            self.idle()                          # periodic checks go here
        else:
            self.handle(message)                 # the module's real job
    except Exception as exc:
        log.exception("%s failed handling %s", self.name, type(message).__name__)
        self.bus.publish(ModuleError(self.name, f"{type(exc).__name__}: {exc}"))

def run(self, stop: threading.Event) -> None:
    while not stop.is_set():
        self.run_once()
```

- **The sentinel `_NOTHING`**: a unique object meaning "nothing arrived". `None` cannot play that role because `None` could be a real message. A brand-new `object()` is equal to nothing else in the program.
- **`threading.Event`** (not a bus event): a thread-safe on/off flag. Setting `stop` tells every loop to finish its current pass and end.
- **Why a 1-second timeout**: an idle worker still loops every second, so it notices `stop` quickly and keeps its heartbeat fresh.
- **`except Exception`**: a bug in `handle` is logged with its traceback, announced as `ModuleError`, and the loop carries on with the next message.

Watch out

`except Exception` catches errors but not `BaseException`s such as `SystemExit`. Those escape, the thread dies, and that is the supervisor's job (Section 9). Also: never put words you heard into an exception message. Tracebacks go to the log, and the log must stay free of spoken words.

Watch out

The error storm (found by the review). The terminal shell listens to *every* event, including `ModuleError`. If printing ever fails (the console closed), the shell fails on an event, publishes a `ModuleError`, receives it, fails to print *that*, publishes another… The reviewer measured about 100,000 error reports in one second, enough to rotate away the whole log. The fix is one condition: failing while handling a `ModuleError` is logged but does not publish a new one. Lesson: any part that both *reports* errors and *listens to* error reports needs a brake.

Daemon threads. Worker threads are started with `daemon=True`: when the main program ends, they end with it. A thread that is truly stuck cannot block Listen from exiting.

7 The event bus: publish and subscribe

```
def publish(self, event: Event) -> None:
    with self._lock:
        inboxes = []
        for cls in type(event).__mro__:      # the class, its parents, ..., Event, object
            for inbox in self._subscribers.get(cls, ()):
                if inbox not in inboxes:
                    inboxes.append(inbox)
    for inbox in inboxes:
        inbox.put(event)
```

Concept: MRO (method resolution order)

Every class knows its family tree: `ModuleError.__mro__` is `(ModuleError, Event, object)`. Walking it means a subscription to `Event` receives every kind of event. That is how the terminal shell prints everything with one line: `bus.subscribe(Event, self.inbox)`, while a module that only cares about `Transcribed` subscribes to just that.

Analogy

A newsletter office. Modules sign up for topics ("ModuleError", "everything"). When a story comes in, the office looks up who signed up for that topic *or any broader topic*, and drops one copy in each person's mailbox. Nobody gets the same story twice, and the writer never needs to know who reads it.

8 Locks and race conditions

Concept: race condition

Two threads touching the same data at the same moment, where the result depends on who got there first. Example: the shell thread subscribes (adding to a list) while the voice thread publishes (reading the same list). Without care, the reader can see a half-changed list.

A `threading.Lock` lets only one thread at a time into a `with self._lock:` block. `publish` *collects* the inboxes inside the lock and *delivers* outside it, so the lock is held for microseconds and never while waiting. Two tests hammer this: 8 threads publishing 500 events each (all 4,000 must arrive), and 200 subscriptions made while another thread publishes non-stop.

9 The supervisor

Every 5 seconds the supervisor looks at each worker:

```
if not self.threads[worker.name].is_alive():          # dead
    if not self._may_restart(worker.name, now):
        self.on_fatal(f"{worker.name} keeps crashing")  # 4th death in a minute
        return
    self._start_thread(worker)                          # new thread, same worker + inbox
elif now - worker.last_beat > HUNG_AFTER:              # alive but silent for 30 s
    self.on_fatal(f"{worker.name} stopped responding")
```

- **A Python thread cannot be restarted**, so a dead worker gets a *new* thread running the *same* worker object. The inbox belongs to the worker, so waiting messages survive.
- **Restart window**: a `deque` (a list that is cheap to trim from the front) holds the times of recent restarts. Times older than 60 seconds are dropped; if 3 remain, a fourth death is a crash loop.
- **Hung**: Python cannot safely kill a thread, so the only cure is restarting the whole agent: `on_fatal` sets exit code 3 and the launcher takes over.

Sleep is not a hang (found by the review). On Windows, `time.monotonic` keeps counting while the laptop sleeps. Close the lid for an hour, open it, and the first check could see every heartbeat as an hour old: "stopped responding", agent stopped. But the supervisor itself checks every 5 seconds, so if more than 30 seconds passed *between two of its own checks*, the whole laptop must have been asleep. In that case it logs "clock jumped" and refreshes every heartbeat instead of blaming the workers. A worker that is really stuck is still caught 30 seconds later.

10 Dependency injection and fakes

How do you test "silent for 30 seconds" without waiting 30 seconds? You do not let the code read the real clock directly. You *hand it* a clock:

```
class Worker:
    def __init__(self, bus, clock=time.monotonic): ...   # real clock by default

class FakeClock:                                          # tests/conftest.py
    def __init__(self, start=1000.0): self.now = start
    def __call__(self): return self.now
    def advance(self, seconds): self.now += seconds

# in a test:
clock.advance(31)
supervisor.check()
assert fatal == ["stuck stopped responding"]
```

Concept: dependency injection

Passing a part's dependencies in (a clock, a place to print, a bus) instead of letting it reach out for them. In normal use the defaults are the real things (`time.monotonic`, `print`); in tests you pass fakes. The terminal shell takes `out=print` for the same reason: tests pass `out=lines.append` and then read the list.

`time.monotonic` not `time.time`: the monotonic clock only ever moves forward. If Windows adjusts the wall clock (daylight saving, an internet time sync), measured durations stay correct.

11 pytest tools used in stage 1

Tool	What it does, and where we used it
fixture	A function that prepares something a test needs; a test asks for it by naming it as a parameter. `bus`, `clock`, `stop`.
`yield` in a fixture	Code after `yield` runs after the test: clean-up. `log_file` removes its log handler; `stop` sets the flag so test threads finish.
`conftest.py`	Fixtures in it are available to every test in the folder, without importing. The fake clock lives there.
`tmp_path`	A fresh, empty folder per test, deleted later. Settings and log files are written there, never in the real `agent/`.
`capsys`	Captures what was printed, to check `--version` and the help text.
`caplog`	Captures log records, to check "recorder failed" was logged.
`monkeypatch`	Temporarily replaces something; undone after the test. `main(["run"])` calls a stand-in `run_agent`, not the real agent.
`parametrize`	Runs one test with many inputs: 8 bad settings files, one test function.
`pytest.raises`	Passes only if the code raises the expected error.

Watching it fail. Every task began with a test run that failed with `ModuleNotFoundError`: the right reason, because the code did not exist yet. A test that passes before the code exists is testing nothing.

12 Logging

```
LOG_FORMAT = "%(asctime)s %(levelname)-5s %(name)-8s %(message)s"
handler = RotatingFileHandler(log_dir / "listen.log", maxBytes=1_048_576, backupCount=5,
                              encoding="utf-8")
```

- A **logger** is what code writes to: `log = logging.getLogger("core")`. Its name becomes the module column.
- A **handler** decides where lines go (here a file); a **formatter** decides how they look. `-5s` and `-8s` pad to 5 and 8 characters so columns line up.
- **Rotation**: at 1 MB, `listen.log` becomes `listen.log.1`, and so on up to `.5`; the oldest is deleted. The log never grows past about 6 MB.

A real line from the first run:

```
2026-10-21 15:09:37,732 INFO  core     ready (version 0.1.0, mode wake)
```

The `words()` rule. Anything you said is wrapped before logging:

```
log.info("heard %s", words(text))
```

`words()` returns `[words hidden]` unless debug is on. The flag is a plain module variable that starts `False` in every new process, which is exactly "debug lasts until restart". A test logs "search what is an atom" and fails if "atom" reaches the file.

13 ruff and CI

`ruff check agent` reads the code without running it. The rule families switched on:

Code	Catches
E	Style errors from PEP 8, Python's style guide (e.g. lines over 100 characters).
F	Real mistakes: unused imports, undefined names (Pyflakes).
I	Imports out of order (standard library, then other libraries, then ours).
B	Likely bugs (flake8-bugbear), e.g. a list as a default argument.
UP	Old syntax a newer Python can write better (pyupgrade), e.g. `int | float`.

`.github/workflows/ci.yml` runs the same `ruff check agent` and `pytest agent` on a fresh Windows machine on every push. If either fails, the commit gets a red cross on GitHub. The server and web jobs are added in stages 4 and 5, when those programs have code.

14 Glossary

argparse
: Python's standard library for reading command-line arguments.

Daemon thread
: A thread that ends automatically when the main program ends.

Dataclass
: A class whose `__init__`, equality and printing are generated from its fields.

Dependency injection
: Handing a part what it depends on (clock, output) instead of letting it fetch it.

Editable install
: `pip install -e`: the installed package points at your source folder.

Exit code
The number a program ends with; 0 is success.

Extra
An optional group of dependencies, e.g. `[dev]`.

Fixture
A pytest helper that prepares (and cleans up) something tests need.

Frozen
A dataclass whose fields cannot be changed after creation.

Handler / formatter
Where log lines go / how they look.

Heartbeat
A timestamp a worker updates on every loop pass, proving it is alive.

Lock Lets only one thread at a time run a block of code.

Monotonic clock
A clock that only moves forward; right for measuring durations.

MRO
A class's family tree, from itself up to `object`.

Queue
A thread-safe waiting line; each worker's inbox.

Race condition
A bug that depends on which thread gets somewhere first.

Sentinel
A unique object used as a marker, e.g. "nothing arrived".

Subcommand
One action of a multi-action program: `listen run`.

Thread
A second line of execution inside the same program.

15 Check your understanding

Check yourself

1. You change `settings.yaml` to `thresholds: { match: 80 }`. What happens when you run `listen run`, and what exit code do you get?
2. Why can't the worker use `None` to mean "no message arrived"?
3. Why is `created_at` filled with `default_factory=_now` and not `= datetime.now(UTC)`?
4. A worker's `handle` raises `ValueError` on every message. Does the supervisor restart its thread? Why or why not?
5. How does the test for "silent for 30 seconds" finish in a fraction of a second?
6. The terminal shell subscribes to `Event`. Why does it receive a `ModuleError`?
7. Why does `publish` put events into inboxes *after* leaving the `with self._lock:` block?
8. What would go wrong if `agent/listen_agent.egg-info/` were committed?